

Tarea 8: Sistema de Gestión de Biblioteca

Semana 8: Programación Orientada a Objetos

NLPY-1: Fundamentos de Python

nablabs

1. Introducción

La **Programación Orientada a Objetos (POO)** es uno de los paradigmas más importantes y utilizados en el desarrollo de software moderno. Permite modelar el mundo real mediante objetos que contienen datos (atributos) y comportamientos (métodos), facilitando la organización, reutilización y mantenimiento del código.

Python es un lenguaje multiparadigma que soporta POO de manera elegante y poderosa. En esta semana aprenderás a diseñar sistemas complejos usando clases, herencia, encapsulamiento, polimorfismo y otros conceptos fundamentales de la POO.

1.1. ¿Por qué Programación Orientada a Objetos?

- **Modelado natural:** Representa entidades del mundo real como objetos
- **Reutilización de código:** Herencia y composición evitan duplicación
- **Encapsulamiento:** Protege datos y controla el acceso a ellos
- **Modularidad:** Código organizado en unidades lógicas independientes
- **Mantenibilidad:** Cambios localizados, menos propenso a errores
- **Polimorfismo:** Mismo comportamiento con diferentes implementaciones

1.2. Objetivos de Aprendizaje

Al completar esta tarea, el estudiante será capaz de:

- Diseñar e implementar clases con atributos y métodos
- Aplicar el método constructor `__init__()`
- Implementar métodos especiales (`__str__`, `__repr__`, etc.)
- Utilizar propiedades con decoradores `@property` y setters
- Aplicar herencia para reutilizar y extender funcionalidad
- Implementar polimorfismo mediante herencia
- Usar métodos de clase (`@classmethod`) y estáticos (`@staticmethod`)
- Aplicar encapsulamiento con atributos privados y protegidos
- Comprender y usar composición de objetos
- Diseñar jerarquías de clases coherentes

2. Descripción del Proyecto

Desarrollarás un **Sistema de Gestión de Biblioteca** completo que permita administrar libros, usuarios, préstamos y multas usando POO. El sistema debe implementar múltiples clases relacionadas con herencia, composición y polimorfismo.

2.1. Funcionalidades Requeridas

2.1.1. 1. Sistema de Clases (40 puntos - Funcionalidad)

Tu sistema debe implementar la siguiente jerarquía de clases:

A) Clase Base: Persona (5 puntos)

- Atributos: nombre, identificación, email, teléfono
- Métodos: `__init__`, `__str__`, `__repr__`
- Validación de datos en el constructor
- Método para actualizar información de contacto

B) Clases Derivadas de Persona (8 puntos)

- **Usuario:** estudiante o miembro regular
 - Atributos adicionales: `tipo_usuario`, `libros_prestados`, `historial`
 - Límite de préstamos según tipo (estudiante: 3, regular: 5)
 - Método para verificar si puede prestar más libros
- **Bibliotecario:** empleado de la biblioteca
 - Atributos: `número_empleado`, `turno`, `especialidad`
 - Métodos para gestionar préstamos y multas

C) Clase Libro (8 puntos)

- Atributos: ISBN, título, autor, editorial, año, categoría
- Atributos de estado: `disponible`, `prestado_a`, `fecha_prestamo`
- Métodos: `prestar`, `devolver`, `obtener_info`
- Propiedades: `@property` para `estado` y `días_prestado`
- Validación de ISBN

D) Clase Prestamo (7 puntos)

- Atributos: `usuario`, `libro`, `fecha_prestamo`, `fecha_devolucion`, `devuelto`
- Calcular días de préstamo y días de retraso
- Método para registrar devolución
- Calcular multa por retraso (C\$500 por día)
- Usar composición (tiene Usuario y Libro)

E) Clase Biblioteca (12 puntos)

- Gestiona colecciones de libros, usuarios y préstamos
- Métodos CRUD para libros y usuarios
- Registrar préstamos y devoluciones
- Buscar libros por título, autor, ISBN o categoría
- Generar reportes y estadísticas
- Listar libros disponibles/prestados
- Calcular total de multas pendientes

2.1.2. 2. Requisitos Técnicos**Uso de POO:**

- Implementar herencia correctamente
- Usar `super()` para llamar constructores padre
- Implementar al menos 3 métodos especiales (`--str--`, `--repr--`, `--eq--`)
- Usar `@property` y setters para atributos calculados
- Implementar al menos un `@classmethod` y un `@staticmethod`

Encapsulamiento:

- Atributos privados (`--`) para datos sensibles
- Atributos protegidos (`_`) para datos internos
- Getters y setters con validación
- No acceder directamente a atributos desde fuera

Diseño:

- Jerarquía de clases coherente y lógica
- Responsabilidad única por clase
- Composición sobre herencia donde sea apropiado
- Métodos bien nombrados y documentados

3. Conceptos de POO en Python

3.1. Elementos Fundamentales

Clases y Objetos:

```
1 class Libro:
2     # Constructor
3     def __init__(self, titulo, autor, isbn):
4         self.titulo = titulo
5         self.autor = autor
6         self.isbn = isbn
7         self.disponible = True
8
9     # Metodo de instancia
10    def prestar(self):
11        if self.disponible:
12            self.disponible = False
13            return True
14        return False
15
16    # Metodo especial para representacion
17    def __str__(self):
18        return f"{self.titulo} por {self.autor}"
19
20    # Crear objetos (instancias)
21    libro1 = Libro("1984", "George Orwell", "978-0451524935")
22    print(libro1) # Usa __str__
```

Listing 1: Definición básica de clase

Herencia:

```
1 class Persona:
2     def __init__(self, nombre, identificacion):
3         self.nombre = nombre
4         self.identificacion = identificacion
5
6     def presentarse(self):
7         return f"Hola, soy {self.nombre}"
8
9 class Usuario(Persona): # Usuario hereda de Persona
10    def __init__(self, nombre, identificacion, tipo):
11        super().__init__(nombre, identificacion) # Llamar constructor padre
12        self.tipo = tipo
13        self.libros_prestados = []
14
15    def presentarse(self): # Sobreescritura (override)
16        return f"{super().presentarse()}, usuario {self.tipo}"
```

Listing 2: Herencia de clases

Propiedades:

```
1 class Libro:
2     def __init__(self, titulo, precio):
3         self.titulo = titulo
4         self._precio = precio # Atributo protegido
5
```

```

6     @property
7     def precio(self):
8         """Getter para precio"""
9         return self._precio
10
11    @precio.setter
12    def precio(self, valor):
13        """Setter con validacion"""
14        if valor < 0:
15            raise ValueError("Precio no puede ser negativo")
16        self._precio = valor
17
18    @property
19    def precio_con_iva(self):
20        """Propiedad calculada (solo lectura)"""
21        return self._precio * 1.13
22
23    # Uso
24    libro = Libro("Python 101", 15000)
25    print(libro.precio) # Usa getter
26    libro.precio = 18000 # Usa setter
27    print(libro.precio_con_iva) # Propiedad calculada

```

Listing 3: Uso de @property

4. Plantilla de Código

4.1. Estructura Base del Sistema

```

1  from datetime import datetime, timedelta
2  from typing import List, Optional
3
4  # =====
5  # CLASE BASE: PERSONA
6  # =====
7
8  class Persona:
9      """Clase base para representar una persona."""
10
11     def __init__(self, nombre: str, identificacion: str,
12                  email: str, telefono: str):
13         """
14         Constructor de Persona.
15
16         Args:
17             nombre: Nombre completo
18             identificacion: Cedula o ID
19             email: Correo electronico
20             telefono: Numero de telefono
21         """
22         # TODO: Validar que los datos no esten vacios
23         # TODO: Asignar atributos
24         pass
25
26     def __str__(self) -> str:

```

```

27     """Representacion en string."""
28     # TODO: Retornar representacion legible
29     pass
30
31     def __repr__(self) -> str:
32         """Representacion tecnica."""
33         # TODO: Retornar representacion para debugging
34         pass
35
36     def actualizar_contacto(self, email: str = None,
37                             telefono: str = None):
38         """Actualiza informacion de contacto."""
39         # TODO: Actualizar email y/o telefono si se proveen
40         pass
41
42 # =====
43 # CLASES DERIVADAS DE PERSONA
44 # =====
45
46 class Usuario(Persona):
47     """Usuario de la biblioteca."""
48
49     LIMITE_PRESTAMOS = {
50         'estudiante': 3,
51         'regular': 5
52     }
53
54     def __init__(self, nombre: str, identificacion: str,
55                 email: str, telefono: str, tipo_usuario: str):
56         """
57         Constructor de Usuario.
58
59         Args:
60             tipo_usuario: 'estudiante' o 'regular'
61         """
62         # TODO: Llamar constructor de Persona con super()
63         # TODO: Validar tipo_usuario
64         # TODO: Inicializar libros_prestados como lista vacia
65         # TODO: Inicializar historial como lista vacia
66         pass
67
68     @property
69     def puede_prestar(self) -> bool:
70         """Verifica si el usuario puede prestar mas libros."""
71         # TODO: Comparar cantidad de libros con el limite
72         pass
73
74     @property
75     def cantidad_prestamos(self) -> int:
76         """Retorna cantidad de libros actualmente prestados."""
77         # TODO: Retornar longitud de libros_prestados
78         pass
79
80     def agregar_prestamo(self, libro):
81         """Agrega un libro a los prestados."""
82         # TODO: Agregar a libros_prestados

```

```

83         pass
84
85     def remover_prestamo(self, libro):
86         """Remueve un libro de los prestados."""
87         # TODO: Remover de libros_prestados
88         pass
89
90 class Bibliotecario(Persona):
91     """Empleado bibliotecario."""
92
93     def __init__(self, nombre: str, identificacion: str,
94                 email: str, telefono: str,
95                 numero_empleado: str, turno: str):
96         """Constructor de Bibliotecario."""
97         # TODO: Llamar constructor de Persona
98         # TODO: Asignar numero_empleado y turno
99         pass
100
101     def registrar_prestamo(self, prestamo) -> bool:
102         """Registra un prestamo en el sistema."""
103         # TODO: Validar y registrar prestamo
104         pass
105
106     def registrar_devolucion(self, prestamo) -> dict:
107         """Registra una devolucion y calcula multa."""
108         # TODO: Procesar devolucion
109         # TODO: Calcular multa si hay retraso
110         pass
111
112 # =====
113 # CLASE LIBRO
114 # =====
115
116 class Libro:
117     """Representa un libro en la biblioteca."""
118
119     def __init__(self, isbn: str, titulo: str, autor: str,
120                 editorial: str, a_o: int, categoria: str):
121         """Constructor de Libro."""
122         # TODO: Validar ISBN (formato basico)
123         # TODO: Asignar todos los atributos
124         # TODO: Inicializar disponible = True
125         # TODO: Inicializar prestado_a = None
126         # TODO: Inicializar fecha_prestamo = None
127         pass
128
129     @property
130     def disponible(self) -> bool:
131         """Retorna si el libro esta disponible."""
132         # TODO: Retornar estado de disponibilidad
133         pass
134
135     @property
136     def dias_prestado(self) -> int:
137         """Calcula dias que lleva prestado el libro."""
138         # TODO: Si esta prestado, calcular dias desde fecha_prestamo

```

```

139         # TODO: Si no, retornar 0
140         pass
141
142     def prestar(self, usuario) -> bool:
143         """Presta el libro a un usuario."""
144         # TODO: Verificar que este disponible
145         # TODO: Marcar como no disponible
146         # TODO: Asignar usuario y fecha
147         pass
148
149     def devolver(self) -> bool:
150         """Marca el libro como devuelto."""
151         # TODO: Marcar como disponible
152         # TODO: Limpiar prestado_a y fecha_prestamo
153         pass
154
155     def __str__(self) -> str:
156         """Representacion en string."""
157         # TODO: Retornar info del libro
158         pass
159
160     def __eq__(self, otro) -> bool:
161         """Compara libros por ISBN."""
162         # TODO: Comparar ISBN de ambos libros
163         pass
164
165     @staticmethod
166     def validar_isbn(isbn: str) -> bool:
167         """Valida formato basico de ISBN."""
168         # TODO: Verificar longitud y formato
169         pass
170
171     # =====
172     # CLASE PRESTAMO
173     # =====
174
175     class Prestamo:
176         """Representa un prestamo de libro."""
177
178         DIAS_LIMITE = 14 # 2 semanas
179         MULTA_POR_DIA = 500 # Colones
180
181         def __init__(self, usuario: Usuario, libro: Libro):
182             """Constructor de Prestamo."""
183             # TODO: Asignar usuario y libro
184             # TODO: Asignar fecha_prestamo (datetime.now())
185             # TODO: Calcular fecha_devolucion (fecha + DIAS_LIMITE)
186             # TODO: Inicializar devuelto = False
187             # TODO: Inicializar fecha_devolucion_real = None
188             pass
189
190         @property
191         def dias_transcurridos(self) -> int:
192             """Calcula dias desde el prestamo."""
193             # TODO: Calcular diferencia entre hoy y fecha_prestamo
194             pass

```

```

195
196 @property
197 def dias_retraso(self) -> int:
198     """Calcula dias de retraso."""
199     # TODO: Si no esta devuelto, calcular retraso desde fecha limite
200     # TODO: Si esta devuelto, calcular desde fecha real
201     # TODO: Retornar 0 si no hay retraso
202     pass
203
204 @property
205 def multa(self) -> float:
206     """Calcula la multa por retraso."""
207     # TODO: Multiplicar dias_retraso por MULTA_POR_DIA
208     pass
209
210 def registrar_devolucion(self) -> dict:
211     """Registra la devolucion del libro."""
212     # TODO: Marcar como devuelto
213     # TODO: Asignar fecha_devolucion_real
214     # TODO: Retornar dict con info de multa
215     pass
216
217 def __str__(self) -> str:
218     """Representacion en string."""
219     # TODO: Retornar info del prestamo
220     pass
221
222 # =====
223 # CLASE BIBLIOTECA
224 # =====
225
226 class Biblioteca:
227     """Sistema de gestion de biblioteca."""
228
229     def __init__(self, nombre: str):
230         """Constructor de Biblioteca."""
231         # TODO: Asignar nombre
232         # TODO: Inicializar listas: libros, usuarios, prestamos
233         pass
234
235     # --- Gestion de Libros ---
236
237     def agregar_libro(self, libro: Libro) -> bool:
238         """Agrega un libro al catalogo."""
239         # TODO: Verificar que ISBN no exista
240         # TODO: Agregar a lista de libros
241         pass
242
243     def buscar_libro_por_isbn(self, isbn: str) -> Optional[Libro]:
244         """Busca un libro por ISBN."""
245         # TODO: Buscar en lista de libros
246         pass
247
248     def buscar_libros_por_titulo(self, titulo: str) -> List[Libro]:
249         """Busca libros por titulo (coincidencia parcial)."""
250         # TODO: Filtrar libros que contengan el titulo

```

```
251     pass
252
253     def buscar_libros_por_autor(self, autor: str) -> List[Libro]:
254         """Busca libros por autor."""
255         # TODO: Filtrar libros del autor
256         pass
257
258     def listar_libros_disponibles(self) -> List[Libro]:
259         """Lista todos los libros disponibles."""
260         # TODO: Filtrar libros con disponible=True
261         pass
262
263     # --- Gestion de Usuarios ---
264
265     def registrar_usuario(self, usuario: Usuario) -> bool:
266         """Registra un nuevo usuario."""
267         # TODO: Verificar que identificacion no exista
268         # TODO: Agregar a lista de usuarios
269         pass
270
271     def buscar_usuario(self, identificacion: str) -> Optional[Usuario]:
272         """Busca un usuario por identificacion."""
273         # TODO: Buscar en lista de usuarios
274         pass
275
276     # --- Gestion de Prestamos ---
277
278     def realizar_prestamo(self, identificacion_usuario: str,
279                          isbn: str) -> dict:
280         """Realiza un prestamo de libro."""
281         # TODO: Buscar usuario y libro
282         # TODO: Verificar que usuario puede prestar
283         # TODO: Verificar que libro este disponible
284         # TODO: Crear objeto Prestamo
285         # TODO: Actualizar estado de libro y usuario
286         # TODO: Agregar a lista de prestamos
287         pass
288
289     def realizar_devolucion(self, isbn: str) -> dict:
290         """Procesa la devolucion de un libro."""
291         # TODO: Buscar prestamo activo del libro
292         # TODO: Registrar devolucion
293         # TODO: Actualizar libro y usuario
294         # TODO: Retornar info con multa
295         pass
296
297     # --- Reportes y Estadisticas ---
298
299     def generar_reporte_general(self) -> dict:
300         """Genera reporte general de la biblioteca."""
301         # TODO: Contar total de libros, usuarios, prestamos
302         # TODO: Calcular libros disponibles/prestados
303         # TODO: Calcular total de multas pendientes
304         pass
305
306     def libros_mas_prestados(self, limite: int = 5) -> List[tuple]:
```

```

307     """Retorna los libros mas prestados."""
308     # TODO: Contar prestamos por libro
309     # TODO: Ordenar y retornar top N
310     pass
311
312     @classmethod
313     def crear_biblioteca_ejemplo(cls):
314         """Crea una biblioteca con datos de ejemplo."""
315         # TODO: Crear instancia de Biblioteca
316         # TODO: Agregar libros y usuarios de ejemplo
317         # TODO: Retornar biblioteca poblada
318         pass
319
320     def __str__(self) -> str:
321         """Representacion en string."""
322         # TODO: Retornar info basica de la biblioteca
323         pass
324
325 # =====
326 # MENU INTERACTIVO
327 # =====
328
329 def menu_principal():
330     """Menu principal del sistema."""
331     # TODO: Crear instancia de Biblioteca
332     # TODO: Implementar menu con opciones:
333     #     1. Gestion de libros
334     #     2. Gestion de usuarios
335     #     3. Realizar prestamo
336     #     4. Realizar devolucion
337     #     5. Buscar libros
338     #     6. Reportes
339     #     7. Salir
340     pass
341
342 # =====
343 # MAIN
344 # =====
345
346 if __name__ == "__main__":
347     print("Sistema de Gestion de Biblioteca")
348     print("NLPY-1 - Semana 8: Programacion Orientada a Objetos")
349     menu_principal()

```

Listing 4: Plantilla de clases principales

4.2. Ejemplo: Implementación de la Clase Libro

Como referencia, aquí está la implementación completa de la clase Libro:

```

1 from datetime import datetime
2
3 class Libro:
4     """Representa un libro en la biblioteca."""
5
6     def __init__(self, isbn: str, titulo: str, autor: str,
7                 editorial: str, a o: int, categoria: str):

```

```

8         """Constructor de Libro."""
9         if not self.validar_isbn(isbn):
10             raise ValueError(f"ISBN invalido: {isbn}")
11
12         self.isbn = isbn
13         self.titulo = titulo
14         self.autor = autor
15         self.editorial = editorial
16         self.a o = a o
17         self.categoria = categoria
18         self._disponible = True
19         self._prestado_a = None
20         self._fecha_prestamo = None
21
22     @property
23     def disponible(self) -> bool:
24         """Retorna si el libro esta disponible."""
25         return self._disponible
26
27     @property
28     def dias_prestado(self) -> int:
29         """Calcula dias que lleva prestado el libro."""
30         if not self._disponible and self._fecha_prestamo:
31             delta = datetime.now() - self._fecha_prestamo
32             return delta.days
33         return 0
34
35     def prestar(self, usuario) -> bool:
36         """Presta el libro a un usuario."""
37         if not self._disponible:
38             return False
39
40         self._disponible = False
41         self._prestado_a = usuario
42         self._fecha_prestamo = datetime.now()
43         return True
44
45     def devolver(self) -> bool:
46         """Marca el libro como devuelto."""
47         if self._disponible:
48             return False
49
50         self._disponible = True
51         self._prestado_a = None
52         self._fecha_prestamo = None
53         return True
54
55     def __str__(self) -> str:
56         """Representacion en string."""
57         estado = "Disponible" if self._disponible else "Prestado"
58         return f"'{self.titulo}' por {self.autor} ({self.a o}) - {estado}"
59
60     def __repr__(self) -> str:
61         """Representacion tecnica."""
62         return f"Libro(isbn='{self.isbn}', titulo='{self.titulo}')"
63

```

```

64     def __eq__(self, otro) -> bool:
65         """Compara libros por ISBN."""
66         if not isinstance(otro, Libro):
67             return False
68         return self.isbn == otro.isbn
69
70     @staticmethod
71     def validar_isbn(isbn: str) -> bool:
72         """Valida formato basico de ISBN."""
73         # ISBN-10: 10 digitos
74         # ISBN-13: 13 digitos (con guiones opcionales)
75         isbn_limpio = isbn.replace('-', '').replace(' ', '')
76         return len(isbn_limpio) in [10, 13] and isbn_limpio.isdigit()
77
78 # Ejemplo de uso:
79 libro = Libro(
80     isbn="978-0451524935",
81     titulo="1984",
82     autor="George Orwell",
83     editorial="Signet Classic",
84     a_o=1950,
85     categoria="Ficcion"
86 )
87
88 print(libro) # '1984' por George Orwell (1950) - Disponible
89 print(repr(libro)) # Libro(isbn='978-0451524935', titulo='1984')

```

Listing 5: Ejemplo: Clase Libro completa

5. Ejemplos de Ejecución

5.1. Creación de Objetos y Herencia

```

1 # Crear usuario estudiante
2 estudiante = Usuario(
3     nombre="Maria Rodriguez",
4     identificacion="1-2345-6789",
5     email="maria.rodriguez@estudiante.cr",
6     telefono="8888-8888",
7     tipo_usuario="estudiante"
8 )
9
10 print(estudiante)
11 # Salida: Maria Rodriguez (1-2345-6789) - Usuario estudiante
12
13 # Verificar limite de prestamos
14 print(f"Puede prestar: {estudiante.puede_prestar}") # True
15 print(f"Prestamos actuales: {estudiante.cantidad_prestamos}") # 0
16
17 # Crear bibliotecario
18 bibliotecario = Bibliotecario(
19     nombre="Carlos Mora",
20     identificacion="3-456-789012",
21     email="cmora@biblioteca.cr",
22     telefono="2222-2222",

```

```
23     numero_empleado="BIB-001",
24     turno="matutino"
25 )
26
27 print(bibliotecario)
28 # Salida: Carlos Mora (3-456-789012) - Bibliotecario BIB-001
```

Listing 6: Ejemplo de creación de usuarios y bibliotecarios

5.2. Gestión de Libros y Préstamos

```
1  # Crear biblioteca
2  biblioteca = Biblioteca("Biblioteca Central nablabs")
3
4  # Agregar libros
5  libro1 = Libro(
6      isbn="978-0451524935",
7      titulo="1984",
8      autor="George Orwell",
9      editorial="Signet Classic",
10     a_o=1950,
11     categoria="Ficcion"
12 )
13
14 libro2 = Libro(
15     isbn="978-0062315007",
16     titulo="Sapiens",
17     autor="Yuval Noah Harari",
18     editorial="Harper",
19     a_o=2015,
20     categoria="Historia"
21 )
22
23 biblioteca.agregar_libro(libro1)
24 biblioteca.agregar_libro(libro2)
25
26 # Registrar usuario
27 biblioteca.registrar_usuario(estudiante)
28
29 # Realizar prestamo
30 resultado = biblioteca.realizar_prestamo(
31     identificacion_usuario="1-2345-6789",
32     isbn="978-0451524935"
33 )
34
35 print(resultado)
36 # Salida:
37 # {
38 #     'exito': True,
39 #     'mensaje': 'Prestamo realizado exitosamente',
40 #     'libro': '1984',
41 #     'usuario': 'Maria Rodriguez',
42 #     'fecha_devolucion': '2024-11-07',
43 #     'dias_limite': 14
44 # }
```

```
46 # Verificar estado
47 print(f"Libro disponible: {libro1.disponible}") # False
48 print(f"Dias prestado: {libro1.dias_prestado}") # 0 (recien prestado)
49 print(f"Usuario puede prestar: {estudiante.puede_prestar}") # True (2/3)
```

Listing 7: Ejemplo de operaciones con libros

5.3. Devolución y Cálculo de Multas

```
1 # Simular que pasaron 20 dias (14 dias limite + 6 de retraso)
2 # En la practica esto seria automatico por las fechas
3
4 resultado = biblioteca.realizar_devolucion(isbn="978-0451524935")
5
6 print(resultado)
7 # Salida:
8 # {
9 #     'exito': True,
10 #     'mensaje': 'Devolucion registrada',
11 #     'libro': '1984',
12 #     'dias_prestamo': 20,
13 #     'dias_retraso': 6,
14 #     'multa': 3000, # 6 dias * 500
15 #     'tiene_multa': True
16 # }
17
18 # Libro vuelve a estar disponible
19 print(f"Libro disponible: {libro1.disponible}") # True
```

Listing 8: Ejemplo de devolución con multa

5.4. Búsquedas y Consultas

```
1 # Buscar libros por titulo
2 resultados = biblioteca.buscar_libros_por_titulo("sapiens")
3 for libro in resultados:
4     print(libro)
5 # Salida: 'Sapiens' por Yuval Noah Harari (2015) - Disponible
6
7 # Buscar por autor
8 resultados = biblioteca.buscar_libros_por_autor("Orwell")
9 print(f"Libros de Orwell: {len(resultados)}")
10 # Salida: Libros de Orwell: 1
11
12 # Listar disponibles
13 disponibles = biblioteca.listar_libros_disponibles()
14 print(f"\nLibros disponibles: {len(disponibles)}")
15 for libro in disponibles:
16     print(f"    - {libro.titulo}")
17 # Salida:
18 # Libros disponibles: 2
19 #    - 1984
20 #    - Sapiens
```

Listing 9: Ejemplo de búsquedas

5.5. Reportes y Estadísticas

```

1  # Generar reporte general
2  reporte = biblioteca.generar_reporte_general()
3
4  print("\n" + "="*60)
5  print(f"REPORTE DE BIBLIOTECA: {biblioteca.nombre}")
6  print("="*60)
7  print(f"Total de libros: {reporte['total_libros']}")
8  print(f"Libros disponibles: {reporte['libros_disponibles']}")
9  print(f"Libros prestados: {reporte['libros_prestados']}")
10 print(f"Total de usuarios: {reporte['total_usuarios']}")
11 print(f"Prestamos activos: {reporte['prestamos_activos']}")
12 print(f"Multas pendientes:      {reporte['multas_pendientes']:,.2f}")
13 print("="*60)
14
15 # Salida:
16 # =====
17 # REPORTE DE BIBLIOTECA: Biblioteca Central nablabs
18 # =====
19 # Total de libros: 15
20 # Libros disponibles: 12
21 # Libros prestados: 3
22 # Total de usuarios: 8
23 # Prestamos activos: 3
24 # Multas pendientes:  4 ,500.00
25 # =====
26
27 # Libros mas prestados
28 top_libros = biblioteca.libros_mas_prestados(limite=3)
29 print("\nTop 3 Libros Mas Prestados:")
30 for i, (libro, cantidad) in enumerate(top_libros, 1):
31     print(f"{i}. {libro.titulo} - {cantidad} prestamos")
32 # Salida:
33 # Top 3 Libros Mas Prestados:
34 # 1. 1984 - 12 prestamos
35 # 2. Sapiens - 8 prestamos
36 # 3. El Principito - 6 prestamos

```

Listing 10: Ejemplo de generación de reportes

5.6. Uso de Propiedades y Validación

```

1  # Crear libro con ISBN invalido (manejo de excepcion)
2  try:
3      libro_invalido = Libro(
4          isbn="123", # ISBN muy corto
5          titulo="Libro de Prueba",
6          autor="Autor Desconocido",
7          editorial="Editorial X",
8          a o=2024,
9          categoria="Prueba"
10     )
11 except ValueError as e:
12     print(f"Error: {e}")
13 # Salida: Error: ISBN invalido: 123

```

```

14
15 # Usar propiedades calculadas
16 libro = Libro(
17     isbn="978-0747532699",
18     titulo="Harry Potter y la Piedra Filosofal",
19     autor="J.K. Rowling",
20     editorial="Bloomsbury",
21     año=1997,
22     categoria="Fantasia"
23 )
24
25 # Prestar el libro
26 usuario = Usuario(
27     nombre="Juan Perez",
28     identificacion="2-3456-7890",
29     email="juan@email.com",
30     telefono="7777-7777",
31     tipo_usuario="regular"
32 )
33
34 libro.prestar(usuario)
35
36 # Acceder a propiedades calculadas
37 print(f"Libro disponible: {libro.disponible}") # False (property)
38 print(f"Dias prestado: {libro.dias_prestado}") # Calculado automaticamente
39
40 # Intentar prestar de nuevo (ya prestado)
41 if not libro.prestar(usuario):
42     print("No se puede prestar: libro ya prestado")
43 # Salida: No se puede prestar: libro ya prestado

```

Listing 11: Ejemplo de propiedades y validación

5.7. Ejecución Completa del Menú

```

1  =====
2  Sistema de Gestion de Biblioteca
3  NLPY-1 - Semana 8: Programacion Orientada a Objetos
4  =====
5
6  =====
7  MENU PRINCIPAL
8  =====
9  1. Gestion de libros
10 2. Gestion de usuarios
11 3. Realizar prestamo
12 4. Realizar devolucion
13 5. Buscar libros
14 6. Reportes y estadisticas
15 7. Salir
16
17 Seleccione una opcion: 1
18
19 =====
20 GESTION DE LIBROS
21 =====

```

```
22 1. Agregar libro
23 2. Buscar libro por ISBN
24 3. Listar todos los libros
25 4. Listar libros disponibles
26 5. Volver
27
28 Seleccione una opcion: 1
29
30 --- Agregar Nuevo Libro ---
31 ISBN: 978-0451524935
32 Titulo: 1984
33 Autor: George Orwell
34 Editorial: Signet Classic
35 A o: 1950
36 Categoria: Ficcion
37
38 Libro agregado exitosamente!
39 ISBN: 978-0451524935
40 Titulo: '1984' por George Orwell (1950)
41
42 Presione Enter para continuar...
43
44 =====
45 MENU PRINCIPAL
46 =====
47 ...
48
49 Seleccione una opcion: 3
50
51 --- Realizar Prestamo ---
52 Identificacion del usuario: 1-2345-6789
53 ISBN del libro: 978-0451524935
54
55 Prestamo realizado exitosamente!
56 Usuario: Maria Rodriguez
57 Libro: 1984
58 Fecha de prestamo: 2024-10-24
59 Fecha limite de devolucion: 2024-11-07
60 Dias permitidos: 14
61
62 Prestamos actuales del usuario: 1/3
63
64 =====
65 MENU PRINCIPAL
66 =====
67 ...
68
69 Seleccione una opcion: 6
70
71 =====
72 REPORTES Y ESTADISTICAS
73 =====
74
75 Biblioteca: Biblioteca Central nablabs
76 -----
77 Total de libros: 8
```

```
78 Libros disponibles: 6
79 Libros prestados: 2
80 Total de usuarios: 5
81 Prestamos activos: 2
82 Total historico de prestamos: 23
83 Multas pendientes: 2 ,500.00
84
85 Top 5 Libros Mas Prestados:
86 1. 1984 (George Orwell) - 7 prestamos
87 2. Cien A os de Soledad (Gabriel Garcia Marquez) - 5 prestamos
88 3. El Principito (Antoine de Saint-Exupery) - 4 prestamos
89 4. Sapiens (Yuval Noah Harari) - 3 prestamos
90 5. Don Quijote (Miguel de Cervantes) - 2 prestamos
91
92 Presione Enter para continuar...
```

Listing 12: Ejemplo de interacción con el menú

6. Defensa del Código (60 % de la Nota)

6.1. Aspectos Técnicos a Dominar

1. Fundamentos de POO (15 puntos)

- ¿Qué es una clase? ¿Qué es un objeto/instancia?
- Diferencia entre atributos de clase y de instancia
- ¿Para qué sirve `self`? ¿Por qué es necesario?
- ¿Qué hace el método `__init__`?
- Explicar el concepto de encapsulamiento

2. Herencia y Polimorfismo (15 puntos)

- ¿Qué es herencia? ¿Cuándo usarla?
- ¿Para qué sirve `super()`?
- Diferencia entre sobreescritura (override) y sobrecarga
- ¿Qué es polimorfismo? Ejemplos en tu código
- Herencia simple vs herencia múltiple

3. Métodos Especiales (10 puntos)

- Diferencia entre `__str__` y `__repr__`
- ¿Qué hace `__eq__`? ¿Cuándo se llama?
- Otros métodos especiales: `__len__`, `__getitem__`
- ¿Por qué implementar métodos especiales?

4. Propiedades y Decoradores (10 puntos)

- ¿Qué es `@property`? ¿Por qué usarlo?
- Diferencia entre `property` y atributo normal
- ¿Cómo implementar un setter?
- `@classmethod` vs `@staticmethod` vs método de instancia
- Ventajas de usar propiedades para validación

5. Diseño y Buenas Prácticas (10 puntos)

- Principio de responsabilidad única
- Composición vs herencia: ¿cuándo usar cada una?
- Atributos privados (`__`) vs protegidos (`_`)
- Nomenclatura de clases y métodos
- Documentación de clases con docstrings

6.2. Preguntas de Defensa Típicas

- ".Explica la jerarquía de herencia en tu sistema"
- "¿Por qué Usuario hereda de Persona? ¿Qué gana con eso?"
- "Muéstrame dónde usas polimorfismo en tu código"
- "¿Qué pasa si llamas `libro.prestar()` cuando ya está prestado?"
- ".Explica cómo funciona tu `property dias_retraso`"
- "¿Por qué usaste composición en la clase `Prestamo`?"
- "Modifica tu código para agregar un método que calcule el préstamo promedio"

7. Rúbrica de Evaluación

7.1. Funcionalidad del Código (40 puntos)

Implementación de Clases (40 puntos):

- 5 puntos: Clase `Persona` con atributos y métodos básicos
- 8 puntos: Clases `Usuario` y `Bibliotecario` con herencia correcta
- 8 puntos: Clase `Libro` con propiedades y validación
- 7 puntos: Clase `Prestamo` con cálculos de multa
- 12 puntos: Clase `Biblioteca` con todas las funcionalidades CRUD

7.2. Defensa del Código (60 puntos)

Fundamentos de POO (15 puntos):

- 5 puntos: Explica clases, objetos y self
- 5 puntos: Comprende constructores y atributos
- 5 puntos: Entiende encapsulamiento

Herencia y Polimorfismo (15 puntos):

- 6 puntos: Implementa herencia correctamente
- 5 puntos: Usa `super()` apropiadamente
- 4 puntos: Demuestra polimorfismo

Métodos Especiales (10 puntos):

- 4 puntos: Implementa `__str__` y `__repr__`
- 3 puntos: Implementa `__eq__` u otros
- 3 puntos: Comprende cuándo se llaman

Propiedades y Decoradores (10 puntos):

- 4 puntos: Usa `@property` correctamente
- 3 puntos: Implementa setters con validación
- 3 puntos: Usa `@classmethod` o `@staticmethod`

Diseño y Calidad (10 puntos):

- 3 puntos: Jerarquía coherente y lógica
- 3 puntos: Nombres descriptivos y convenciones
- 2 puntos: Documentación con docstrings
- 2 puntos: Código organizado y limpio

7.3. Penalizaciones

- **-5 puntos:** No usar herencia cuando es apropiado
- **-5 puntos:** No implementar métodos especiales básicos
- **-4 puntos:** No usar `super()` en constructores de clases derivadas
- **-3 puntos:** Acceder directamente a atributos privados
- **-3 puntos:** No validar datos en constructores
- **-2 puntos:** Clases sin docstrings
- **-2 puntos:** Nomenclatura incorrecta (clases no en PascalCase)

8. Desafíos Opcionales (Puntos Extra)

8.1. Nivel Intermedio (+10 puntos cada uno)

1. Sistema de Categorías de Libros

- Crear clase Categoria con atributos y métodos
- Implementar jerarquía de categorías (ficción, no-ficción, etc.)
- Búsqueda por categoría jerárquica

2. Reservas de Libros

- Clase Reserva para cuando libro no está disponible
- Lista de espera por libro
- Notificación automática cuando se devuelve

3. Historial Detallado

- Registrar todo el historial de préstamos
- Estadísticas por usuario (libros más leídos, categorías favoritas)
- Generar reporte de historial

8.2. Nivel Avanzado (+15 puntos cada uno)

4. Sistema de Membresías

- Clases para diferentes tipos de membresía (básica, premium, VIP)
- Beneficios por nivel (días de préstamo, cantidad de libros)
- Cálculo de cuotas y renovaciones

5. Persistencia con JSON

- Guardar y cargar biblioteca desde archivos JSON
- Serialización de objetos
- Métodos de clase para crear desde archivo

6. Sistema de Recomendaciones

- Recomendar libros basados en historial
- Algoritmo simple de similitud
- Top 5 recomendaciones personalizadas

8.3. Nivel Experto (+20 puntos)

7. Patrón Observer para Notificaciones

- Implementar patrón Observer
- Notificar eventos (préstamo, devolución, multa)
- Sistema de suscriptores

8. Interfaz Gráfica con tkinter

- Crear GUI básica para el sistema
- Mantener toda la lógica POO
- Ventanas para cada operación principal

9. Recursos y Referencias

9.1. Documentación

- Tutorial POO oficial: <https://docs.python.org/3/tutorial/classes.html>
- Métodos especiales: <https://docs.python.org/3/reference/datamodel.html>
- Decoradores property: <https://docs.python.org/3/library/functions.html#property>

9.2. Consejos para el Desarrollo

1. **Diseña primero:** Dibuja tu jerarquía de clases antes de programar
2. **Empieza simple:** Implementa clases básicas primero, luego agrega complejidad
3. **Prueba cada clase:** Verifica cada clase individualmente antes de integrar
4. **Usa docstrings:** Documenta cada clase y método importante
5. **Piensa en objetos:** Modela entidades del mundo real
6. **Valida siempre:** No asumas que los datos son correctos

10. Criterios de Entrega

10.1. Archivos Requeridos

1. **biblioteca.py:** Código principal con todas las clases
2. **pruebas.py:** Script con casos de prueba
3. **diagrama_clases.txt:** Diagrama de jerarquía de clases (texto ASCII)
4. **README.txt:** Documentación del proyecto

10.2. Contenido del README

- Nombre del estudiante
- Diagrama de clases (descripción textual)
- Instrucciones de uso
- Ejemplos de operaciones principales
- Desafíos opcionales implementados
- Decisiones de diseño importantes

*”La programación orientada a objetos no es solo sobre sintaxis,
es sobre pensar en términos de responsabilidades y relaciones.”*

— Adaptación de principios de diseño OOP

¡Éxito con tu sistema de biblioteca!

Recuerda: Un buen diseño orientado a objetos hace el código más mantenible y extensible.